

TEXT-TO-SQL CHATBOT

Web Application Execution and Deployment Plan

A practical build plan for converting the current notebook prototype into a polished, publicly shareable internship project.

Target outcome	A Streamlit web application that lets users ask sales questions in plain English, produces safe read-only SQL, displays results clearly, and can be deployed through Streamlit Community Cloud without a paid database.
Deployment architecture	Browser -> Streamlit application -> LLM API (optional key) -> local SQLite sales database. The database is created from the repository CSV files and travels with the application.
Project position today	The repository contains Jupyter notebook experiments and CSV data. It does not yet contain an application entry point, UI, repeatable database setup, dependency manifest, or deployment configuration.

Scope and success criteria

- Deliver a polished public demo rather than a production enterprise system.
- Avoid MySQL hosting costs by using a packaged SQLite database built from the sales CSV files.
- Keep all database access read-only and enforce a limited result size.
- Keep API keys outside GitHub through local environment variables and Streamlit Secrets.
- Make the source repository easy for a reviewer to run locally or deploy from GitHub.

Implementation phases

PHASE 1 | Repository assessment and foundation

Objective: Establish a clean, runnable application structure without modifying the original notebooks.

- Inspect notebook logic, CSV headers, and existing documentation to identify the canonical six-table sales schema.
- Create separate application folders for UI, services, database assets, and tests.
- Add a pinned requirements file, Python version declaration, environment-variable template, and updated .gitignore rules.
- Preserve the notebooks as research artifacts and move all deployable logic into normal Python modules.

Deliverable: A conventional project layout with a single web-app entry point and reproducible dependencies.

Exit check: A fresh Python environment can install the listed packages and start the app command without relying on notebook state.

PHASE 2 | Portable data layer

Objective: Replace local MySQL assumptions with a database that is included with the demo.

- Create a database-build script that imports the selected CSV files into SQLite tables.
- Normalize table names and document table relationships so the SQL prompt matches the actual database.
- Add schema inspection helpers and a small set of verified business questions for demonstration.
- Keep the generated SQLite file out of source control if it becomes large; rebuild it automatically during startup or deployment.

Deliverable: A repeatable SQLite database build and a documented sales-data schema.

Exit check: The database can be deleted and rebuilt from the committed CSV files, and each reference query returns expected rows.

PHASE 3 | Text-to-SQL engine and safety layer

Objective: Provide useful AI query generation while preventing destructive or runaway SQL.

- Create a provider adapter that reads an API key from GOOGLE_API_KEY or a selected alternative provider key.
- Supply the LLM only with the approved database schema and instruct it to generate SQLite-compatible SELECT statements.
- Extract SQL safely from model responses and reject multi-statement queries, comments, DDL, DML, pragma commands, and non-SELECT operations.
- Apply an automatic row limit and present errors in plain language rather than exposing stack traces.
- Add a fallback message and prebuilt example queries so the app remains presentable if an LLM key is unavailable.

Deliverable: A testable service that takes a natural-language question and returns validated SQL plus tabular results.

Exit check: Unsafe statements are blocked, valid sample questions work, and a missing/invalid provider key does not crash the UI.

Implementation phases (continued)

PHASE 4 | User interface and presentation

Objective: Turn the engine into a convincing internship-ready product experience.

- Build a responsive Streamlit page with a clear product name, short value proposition, and a professional color/type hierarchy.
- Add a chat-style question input, a submit action, and clickable suggested questions related to sales, customers, budgets, and regions.
- Display the answer, generated SQL, result count, and sortable results table in clear sections.
- Add a sidebar for the available datasets, safety notice, and app instructions.
- Use loading states, empty states, and friendly error messages so every interaction has a deliberate visual outcome.

Deliverable: A complete browser interface that is suitable for screenshots, a portfolio link, and a live demo.

Exit check: The UI is understandable without opening a notebook; all key actions can be completed from the web page.

PHASE 5 | Quality assurance and documentation

Objective: Verify the application works locally and make it easy for reviewers to understand the engineering choices.

- Run the app in a clean local session and execute the curated sample questions.
- Test invalid questions, unavailable API keys, malformed model responses, no-result queries, and blocked SQL attempts.
- Check the interface at desktop width and narrow browser width for clipping, cramped controls, and unreadable tables.
- Rewrite the README around installation, local execution, architecture, safety decisions, screenshots, and deployment steps.
- Add a concise project-summary section that can be reused in the internship application.

Deliverable: A verified local build, clear README, and ready-to-share GitHub repository.

Exit check: A new user can follow the README to install dependencies, set a secret, and run the app successfully.

PHASE 6 | GitHub and Streamlit Community Cloud deployment

Objective: Publish a public demonstration link without a hosted MySQL service.

- Commit only source code, CSV assets needed by the demo, and configuration templates; never commit API keys or local secret files.
- Push the completed project to a GitHub repository.
- Create a Streamlit Community Cloud app, select the repository and the app entry-point file, then set the Python version if required.
- Add the LLM key in Streamlit Secrets using the same environment-variable name expected by the application.
- Open the public deployment URL, run all sample questions, and review app logs if the startup fails.

Deliverable: A public Streamlit URL connected to the GitHub repository, with secrets stored outside source control.

Exit check: The deployed app loads successfully, uses the packaged SQLite data, and can answer at least the verified example questions.

Deployment checklist and responsibilities

Item	Who completes it	How it is completed
Application source and UI	Implementation work	Build and test the Streamlit app, data layer, safety controls, and project documentation in this folder.
GitHub repository	Account owner	Sign in to GitHub if required and authorize the repository push. Source remains public only after you choose to publish it.
Streamlit deployment	Implementation work + account owner	Create/select the deployment through the signed-in Streamlit account, choose the repository entry point, then confirm the generated public URL.
LLM API key	Account owner	Create or supply a valid key from an accessible provider. Paste it only into Streamlit Secrets; do not send it in chat or commit it to Git.
Final submission	Account owner	Use the verified public URL, GitHub repository URL, and project summary in the internship application.

Final acceptance checklist

- The app launches with one command after dependency installation.
- No password or API key appears in tracked files, notebook outputs, or the public repository.
- The database is generated from source CSV files and no external MySQL server is required.
- Only safe, single-statement read queries are executed.
- The public page has a clean UI, clear instructions, and working sample questions.
- The README includes the live URL, architecture, local setup, and limitations of the free demo deployment.

Recommended project description

Built a web-based natural-language-to-SQL analytics assistant using Python, Streamlit, SQLite, and an LLM. The application converts business questions into validated read-only SQL, queries a multi-table sales dataset, and presents results through a responsive interface designed for non-technical users.